

Aula 2 — C# Moderno para Web

Carga horária: 4 horas

Projeto base: Aula01.Web

Objetivo da aula:

Dominar os principais recursos do **C# moderno (C# 10/11/12)** aplicados ao desenvolvimento web com ASP.NET Core, utilizando **boas práticas de código, injeção de dependência, LINQ avançado e programação assíncrona**.

PARTE 1 — TEORIA (≈ 2 horas)

1 Tipos Modernos em C#

- ◆ `class`
 - Tipo de referência
 - Mutável
 - Ideal para entidades com ciclo de vida longo

```
public class Usuario { public int Id { get; set; } public string Nome { get; set; } }
```

- ◆ `record`
 - Imutável por padrão
 - Comparação por valor
 - Ideal para **DTOs e respostas de API**

```
public record UsuarioDto(int Id, string Nome);
```

- ✓ Comparação automática
 - ✓ Código mais limpo
 - ✓ Excelente para Web
-

◆ struct

- Tipo de valor
- Alocado na stack (quando possível)
- Usado para objetos pequenos e imutáveis

```
public struct Coordenada { public double Latitude { get; } public double Longitude { get; } public Coordenada(double lat, double lng) { Latitude = lat; Longitude = lng; } }
```

👉 Resumo prático

Tipo	Uso recomendado
class	Entidades
record	DTOs
struct	Value Objects

2 LINQ Avançado

- ### ◆ Operações principais
- Where

- Select
- GroupBy
- OrderBy
- Any , All
- First , Single
- Join

Exemplo:

```
var ativos = usuarios .Where(u => u.Ativo) .OrderBy(u => u.Nome) .Select(u => new UsuarioDto(u.Id, u.Nome)) .ToList();
```

3 Async / Await

◆ Por que usar?

- Não bloqueia threads
- Escalabilidade
- Melhor performance web

◆ Conceito

```
public async Task<string> BuscarMensagemAsync() { await Task.Delay(1000); return "Mensagem assíncrona"; }
```

👉 Regra de ouro:

Toda chamada I/O no backend **deve ser async**

4 Dependency Injection (DI) Nativa

ASP.NET Core possui DI **nativa**.

Ciclos de vida:

Tipo	Descrição
Transient	Nova instância sempre
Scoped	Uma por request
Singleton	Única instância

Exemplo:

```
builder.Services.AddScoped<IMensagemService, MensagemService>();
```

5 Logging e Configuration

◆ Logging

- ILogger<T>
- Logs estruturados
- Integração com Application Insights, ELK, etc.

```
_logger.LogInformation("Usuário acessou a página Home");
```

◆ Configuration

Arquivo appsettings.json :

```
{ "AppSettings": { "NomeAplicacao": "Sistema Web .NET" } }
```

Acesso:

```
var nome = _configuration["AppSettings:NomeAplicacao"];
```

PARTE 2 — PRÁTICA (≈ 2 horas)

1 Refatoração do Projeto

Estrutura ajustada

```
Aula01.Web |— Controllers |— Models |   |— Usuario.cs |— DTOs |   |— UsuarioDto.cs |— Services | |— Interfaces | |   |— IUserarioService.cs |   |— UsuarioService.cs
```

2 Criando o Model (Entidade)

 Models/Usuario.cs

```
namespace Aula01.Web.Models { public class Usuario { public int Id { get; set; } public string Nome { get; set; } public bool Ativo { get; set; } } }
```

3 Criando o DTO (record)

 DTOs/UsuarioDto.cs

```
namespace Aula01.Web.DTOs { // DTO imutável usando record public record UsuarioDto(int Id, string Nome); }
```

4 Interface do Serviço

 Services/Interfaces/IUsuarioService.cs

```
using Aula01.Web.DTOs; namespace Aula01.Web.Services.Interfaces { public interface IUsuarioService { Task<List<UsuarioDto>> ObterUsuariosAtivosAsync(); } }
```

5 Implementação do Serviço (LINQ + Async)

 Services/UsuarioService.cs

```
using Aula01.Web.DTOs; using Aula01.Web.Models; using Aula01.Web.Services.Interfaces; namespace Aula01.Web.Services { public class UsuarioService : IUsuarioService { // Simulando base de dados em memória private readonly List<Usuario> _usuarios = new() { new Usuario { Id = 1, Nome = "Ana", Ativo = true }, new Usuario { Id = 2, Nome = "Bruno", Ativo = false }, new Usuario { Id = 3, Nome = "Carlos", Ativo = true } }; public async Task<List<UsuarioDto>> ObterUsuariosAtivosAsync() { // Simula operação I/O await Task.Delay(300); // LINQ avançado + projeção para DTO return _usuarios .Where(u => u.Ativo) .OrderBy(u => u.Nome) .Select(u => new UsuarioDto(u.Id, u.Nome)) .ToList(); } } }
```

6 Registro do Serviço no Program.cs

```
builder.Services.AddScoped<IUsuarioService, UsuarioService>();
```

7 Uso no Controller com DI + Logging

 Controllers/HomeController.cs

```
using Aula01.Web.Services.Interfaces; using Microsoft.AspNetCore.Mvc; namespace Aula01.Web.Controllers { public class HomeController : Controller { private readonly IUserarioService _usuarioService; private readonly ILogger<HomeController> _logger; public HomeController( IUserarioService usuarioService, ILogger<HomeController> logger) { _usuarioService = usuarioService; _logger = logger; } public async Task<IActionResult> Index() { _logger.LogInformation("Carregando usuários ativos"); var usuarios = await _usuarioService.ObterUsuariosAtivosAsync(); return View(usuarios); } } }
```

8 Atualizando a View Razor

 Views/Home/Index.cshtml

```
@model List<Aula01.Web.DTOs.UsuarioDto> <h2>Usuários Ativos</h2> <ul> @foreach (var usuario in Model) { <li>@usuario.Nome</li> } </ul>
```

✓ Resultado Esperado

- Uso correto de records
 - LINQ aplicado a entidades e DTOs
 - Código assíncrono
 - Serviços injetáveis
 - Logging profissional
 - Projeto já com padrão de mercado
-

Atividade Avaliativa da Aula

Desafio prático:

1. Criar entidade Produto
2. Criar ProdutoDto
3. Criar IProdutoService
4. Retornar apenas produtos ativos com LINQ
5. Exibir na View